

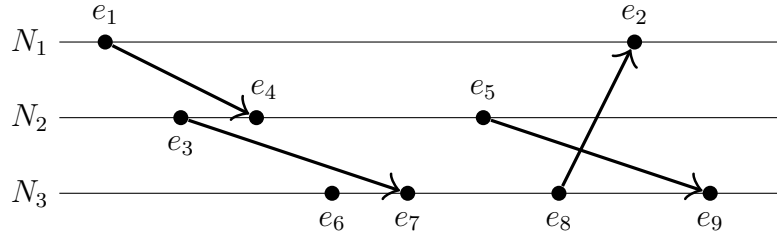
COMPUTER SCIENCE TRIPOS Part IB – 2024 – Paper 5

5 Concurrent and Distributed Systems (tlh20)

Broadcast
protocols and
logical time

Distributed systems often use *logical timestamps* to capture the possible ordering between operations on different nodes.

(a) Three nodes $N_1 \dots N_3$ communicate using point-to-point messages:



(i) Give *Lamport timestamps* for each of the events $e_1 \dots e_9$, assuming clocks start from 0. [2 marks]

Answer:

$e_1 - 1$	$e_3 - 1$	$e_6 - 1$
$e_4 - 2$	$e_7 - 2$	
$e_5 - 3$	$e_8 - 3$	
$e_2 - 4$	$e_9 - 4$	

(ii) What is a necessary condition on Lamport timestamps for an event x to have *happened before* an event y ? Is this a sufficient condition? Briefly justify your answer based on the events in the example. [2 marks]

Answer: If L_x is the Lamport timestamp for event e_x , then $L_x < L_y$ is necessary for e_x to happen before e_y . This is not a sufficient condition—for instance, $L_6 < L_4$ but these events are concurrent.

(iii) Now give possible *vector timestamps* for each of the events $e_1 \dots e_9$. [2 marks]

Answer:

$e_1 - \langle 1, 0, 0 \rangle$	$e_3 - \langle 0, 1, 0 \rangle$	$e_6 - \langle 0, 0, 1 \rangle$
$e_4 - \langle 1, 2, 0 \rangle$	$e_7 - \langle 0, 1, 2 \rangle$	
$e_5 - \langle 1, 3, 0 \rangle$	$e_8 - \langle 0, 1, 3 \rangle$	
$e_2 - \langle 2, 1, 3 \rangle$	$e_9 - \langle 1, 3, 4 \rangle$	

(iv) What is a *necessary and sufficient* condition on vector timestamps for an event x to have happened before an event y ? [2 marks]

Answer: If T_x is the vector timestamp for event e_x , then each element of T_x must be less than or equal to the corresponding element of T_y , and they must differ in at least one element.

- (b) An alternate form of vector timestamps can be used to build *causal broadcast*. That is, if the broadcast of one message happened before the broadcast of another message, then all nodes must deliver those two messages in that order.
- (i) Write pseudo-code to implement causal broadcast, assuming access to an underlying reliable broadcast protocol. You should describe the local state held at each node along with the algorithm used to broadcast a message, and to decide when a message can be delivered locally. [8 marks]

Answer: This algorithm is covered in the lecture on ‘Broadcast protocols and logical time’. The key idea is to use a vector that counts the number of messages that have been received from each node that have been delivered locally. To broadcast a message we include the vector for the messages already delivered from other nodes at the sender. On receiving a message via reliable broadcast, we delay the message until the receiver has all of the messages that were present at the sender when the message was sent.

State:

```
sendSeq = 0
delivered = <0, 0, ... 0>
buffer = {}
```

On request to broadcast message m at node i :

```
deps = delivered
deps[i] = sendSeq
send <i, deps, m> via reliable broadcast
sendSeq = sendSeq + 1
```

On receiving message m via reliable broadcast at node i :

```
buffer = buffer U {m}
while \exists <sender, deps, msg> \in buffer s.t. deps <= delivered:
    deliver msg to the application
    buffer = buffer \ {<sender, deps, msg>}
    delivered[sender] = delivered[sender] + 1
```

- (ii) A colleague suggests that the space required by vector timestamps makes this algorithm inefficient. They suggest basing the algorithm on Lamport timestamps instead. Do you agree with this suggestion? [4 marks]

Answer: As with earlier parts of this question, this final section is based on the lecture on ‘Broadcast protocols and logical time’. However, the suggestion is not covered explicitly in the lectures and requires some analysis of the positive/negative aspects of these different kinds of logical time.

In summary, Lamport-style timestamps are generally unsuitable for use in causal broadcast.

A node could use those timestamps to organise messages it receives to be consistent with causal order. However, delivering a message requires the node to be certain that there are no prior messages that have not yet been delivered. That could be ensured if we have FIFO delivery and delay each message until a subsequent message is seen from every node. However, as with the discussion of simplistic total-order broadcast in the lectures (Slide 86), the failure of any node will then prevent system-wide progress.

— *Solution notes* —

From a practical viewpoint, the benefits of using single values in place of vectors are likely to be small compared with the use of buffering for messages.
